

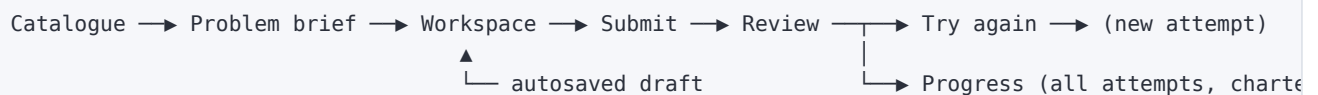
Design note

The MVP, the user flow, the classes that carry the domain, how evaluation works, and what was traded away.

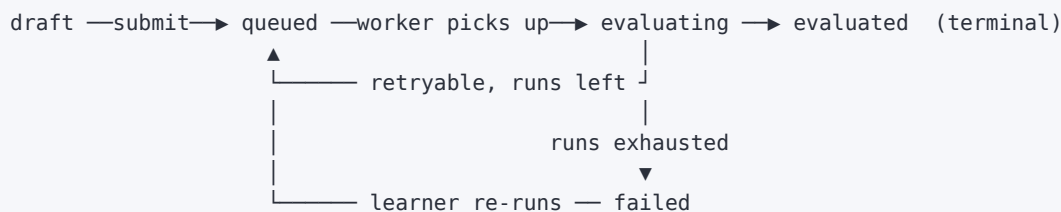
1. The MVP in one paragraph

A learner picks a problem from a seeded catalogue, works on a design in one of three formats, and submits it. Submission returns immediately with the attempt `queued`; a background worker normalises the submission into a common design model, runs it through an evaluation pipeline (deterministic rules always, a language model when available), and stores a scored review. The client polls until the attempt leaves a pending state, then shows the score per rubric dimension, the feedback with each item labelled by its source, and requirement-by-requirement coverage with the matched evidence. Every attempt is kept, so the history view charts score across attempts per problem.

2. User flow



The states behind it are the attempt's own, not a spinner:



3. The domain

Three aggregates, one of them doing most of the work.

Problem : authored content, immutable

Carries the learner-facing brief *and* the grading key: `requirements` (each with `signals`, the synonym set that counts as addressing it), `expectedConcepts` (with aliases and a *why*), `pitfalls` (problem-specific traps with trigger terms), and a `Rubric`. The DTO layer strips the key before it reaches the client.

The design decision here is that a problem is data, not code. Adding "Design a Chess Game" is a new entry in `seedProblems()`: the deterministic checker picks up its signals, concepts and pitfalls with no new classes. Problem authoring is a content task.

Rubric : six fixed dimensions, per-problem weights

`requirement_coverage`, `abstraction_quality`, `relationships`, `extensibility`, `edge_cases`, `tradeoff_reasoning`.

The axes are fixed deliberately: a learner comparing attempt #1 to attempt #4 needs them to mean the same thing every time. Problems tune the weights (Elevator leans on relationships and trade-offs; Vending Machine on edge cases).

Each dimension also carries a `deterministicShare` $\in [0,1]$: how much of that dimension's score comes from rules versus the model. This one number is where the platform's whole evaluation philosophy lives, expressed as data rather than as branching inside the evaluator:

Dimension	Deterministic share	Why
Requirement coverage	0.7	A finite authored checklist. Mostly a fact.
Relationships	0.5	Cardinality present/absent is a fact; whether it is <i>right</i> is judgement.
Edge cases	0.4	Vocabulary is detectable; sufficiency is not.
Abstraction quality	0.3	Crowded classes are detectable; good taste is not.
Extensibility	0.3	Seams are detectable; whether they are the <i>right</i> seams is not.
Trade-off reasoning	0.0	Only a reader can judge an argument.

Attempt : the aggregate root of the practice loop

Owns its status, its submission, its evaluation, its event timeline and its run count. **Every state change goes through a method on this class, and every method consults a single transition table** (`AttemptStatus.ts`). Nothing outside the class assigns to `status`, so "can an evaluated attempt be resubmitted?" has exactly one answer in exactly one file.

Two behaviours worth calling out:

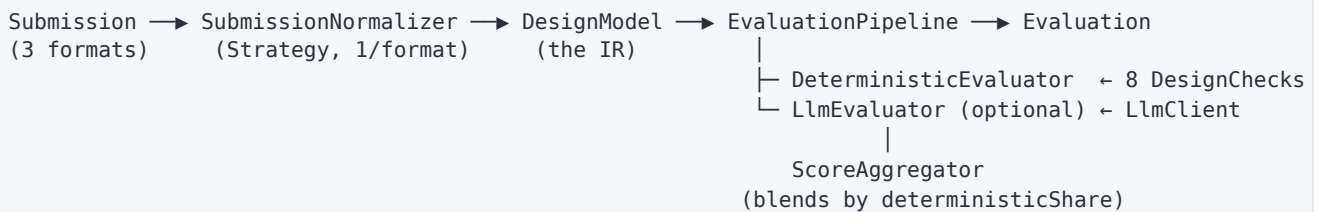
- `submit()` **validates before it transitions**. A submission too thin to be worth evaluating leaves the attempt untouched and still editable: nothing the learner typed is lost. (*This ordering was a real bug, caught by a test; see `AI_USAGE.md`.*)
- `failEvaluation(reason, retryable)` decides between re-queueing and giving up, based on the retryability of the error and the runs remaining. The attempt, not the worker, owns its own retry budget.

Submission : a discriminated union, not a hierarchy

`StructuredSubmission` | `CodeSubmission` | `DiagramSubmission`. Inert data that crosses the wire and the database; behaviour lives in the normalisers that read it.

The product opinion in the MVP is that the structured format is primary. Free prose is pleasant to write, impossible to give precise feedback on, and lets a learner skip naming responsibilities and cardinality: the very things being taught. Code and diagram exist for learners who think in them.

4. The evaluation architecture



DesignModel : the intermediate representation

Entities, relationships, operations, narrative, plus a flattened lower-case `searchText` for signal matching. **Evaluators depend on this and never on the raw submission.** That is what makes the format axis and the evaluator axis independent: a new format is one normaliser and zero evaluator changes; a new evaluator works across all three formats for free.

DesignCheck : the deterministic half

Eight small, independent, synchronous rules: requirement coverage, concept coverage, responsibility distribution, relationship clarity, extensibility seams, edge-case awareness, trade-off articulation, known pitfalls. Each returns a score for one dimension, a rationale, and feedback items.

Handling "more than one valid answer" is a matter of wording and of floors. `ConceptCoverageCheck` never says a concept is missing, it says nothing was *detected* that owns it, and asks whether that was deliberate. It has a score floor, so disagreeing with the reference costs some marks and never all of them. Requirement coverage, where there genuinely is a right answer, has no such floor.

Three refinements came out of running hand-written designs through the platform and finding the verdicts unconvincing:

Concepts are scored three ways, not two. A concept can be *dedicated* (a type is named for it: full credit), *folded* (it appears only inside another type's members, e.g. `calculateFee()` on a god class, half credit and a question), or *absent* (no credit). Grading `folded` the same as `dedicated` let a design that merely name-drops score as well as one that models the concept; grading it the same as `absent` would assert that the reference decomposition is the only correct one. Half credit says what is actually true, the concept is present, but its ownership is unclear from the design alone.

Edge-case awareness is scored from the problem's own implied requirements, not a global word list. Every problem authors the cases the brief deliberately omits (a full lot, two cars racing for the last spot, an equal split of 100 across three people) each with tuned signals. Those are a far sharper instrument than generic failure vocabulary, and they reward noticing the case rather than using a particular word. The generic vocabulary stays as a smaller secondary signal, since a design can be thoughtful in ways the problem's author did not anticipate. Reading the implied requirements here as well as in `RequirementCoverageCheck` is deliberate rather than double counting: that check asks *did you cover the brief*, this one asks *did you think past it*.

A check that finds nothing abstains rather than scoring 100. `PitfallCheck` returns no score at all when a design walks into none of the known traps. Not falling into a handful of named pitfalls is the absence of evidence, not evidence of good abstraction, and scoring it as a perfect result quietly lifted every design's

abstraction average for doing nothing.

This half is the platform's **floor**: dependency-free, single-digit milliseconds, and reproducible. Whatever happens to the model, a learner who submits gets a real score with real evidence.

LlmEvaluator : the qualitative half

Judges only the five dimensions where taste is required, and is **given the deterministic findings in its prompt and told not to repeat them**, so the two halves compose into one review rather than two overlapping ones. Output is strict JSON validated by a Zod schema; the parser scans for the first brace-balanced object (ignoring braces inside strings) because models wrap JSON in prose. Temperature 0.1: the same design resubmitted should get the same review, or the score history means nothing.

Three prompt rules exist because of observed behaviour without them: without *"there is more than one correct design"* it graded by similarity to the reference; without *"quote something specific from their submission"* it produced fluent generic advice; without *"do not comment on requirement coverage"* it duplicated and sometimes contradicted the rules.

ScoreAggregator

Blends per dimension by `deterministicShare`; falls back to whichever source reported when one is missing; **renormalises the overall score over the dimensions actually assessed**, so a degraded run is not silently scored out of a different total.

5. What happens when evaluation is slow or fails

Three levels, and at no level is the learner's submission lost or the error swallowed:

1. **An optional evaluator fails** (the usual case: model rate-limited, timed out, or returned unparseable JSON). The pipeline completes with a deterministic-only review, flags `degraded: true` with the reason, and the UI shows a "Partial review" banner with a re-run button. The learner gets a genuinely useful review, clearly marked as half of one.
2. **The whole run fails retryably**. The worker re-queues with exponential backoff (1s, 2s, 4s), up to the attempt's run limit of 3.
3. **Runs exhausted, or a non-retryable error**. The attempt lands in `failed` with the reason kept and the submission intact; the learner re-runs it from the UI with a fresh budget.

Evaluation runs off a `JobQueue<T>` interface with an in-process implementation. It is honest about being in-process: a restart loses queued jobs, leaving the attempt in `queued` for a re-run. That is an acceptable trade when the submission itself is already durable, and a durable broker is a dependency this MVP has not earned.

6. Extensibility: the four seams, and what each costs

Change	What you write	What you do not touch
A new submission format (image + OCR, repo link, JSON export)	One <code>SubmissionNormalizer</code> , registered in the container	Any evaluator, service, route, or the aggregator
A new evaluation approach (a static analyser over submitted code, a peer-review queue, a second model voting)	One <code>Evaluator</code> , added to the pipeline's optional list	The pipeline, the worker, the attempt lifecycle
A new problem	One entry in <code>seedProblems()</code> : signals, concepts, pitfalls, weights	No code at all
A real queue / database / model provider	One adapter behind <code>JobQueue</code> , <code>AttemptRepository</code> , <code>LlmClient</code>	One line each in <code>container.ts</code>

The composition root (`container.ts`) is the only file in the codebase that names a concrete adapter. Everything else takes its collaborators through its constructor.

7. Key trade-offs

Structured submission as the primary format. *Cost:* more friction than a text box, and it nudges toward a conventional shape. *Why anyway:* the friction is the teaching. A learner who has to write one line of responsibility per class discovers the overloaded class themselves, which is the lesson.

Signal matching rather than parsing meaning. Requirement coverage is keyword-set matching over the design text. *Cost:* it can be gamed by name-dropping, and it can miss a design that solves a requirement in unusual vocabulary. *Why anyway:* it is instant, free, reproducible, and it can show its evidence, which is what makes it arguable rather than authoritative. The model covers what it misses, and the UI shows exactly what matched so a false verdict is visible.

Regex-based code parsing rather than a real AST. *Cost:* it will misread exotic syntax. *Why anyway:* the evaluators need shape, not precision, and the model reads the raw source anyway. Three language parsers is a week of work for a sharper answer to a question already half-answered. `CodeNormalizer` is the single class to replace if code becomes the popular format.

In-process queue. *Cost:* jobs do not survive a restart. *Why anyway:* the assignment explicitly warns against turning this into a distributed-systems project, the submission is durable, and the interface means the upgrade is an adapter.

Attempts are immutable once submitted. *Cost:* a typo means starting a new attempt. *Why anyway:* the history is the product. An editable attempt makes "did I improve?" unanswerable.

No accounts. *Cost:* clearing browser storage loses your history. *Why anyway:* auth is orthogonal to everything being evaluated here, and the learner identity is isolated in one function on each side, so adding it is a small, contained change.

8. If it needed to handle more users or a slower model

Kept brief, per the brief's scope boundary. The two constrained resources are model latency/cost and evaluation throughput: nothing else in this system is heavy.

- **The queue interface is already the seam.** Swap `InProcessJobQueue` for BullMQ (Redis) or a Kafka topic and run the worker as a separate process. No domain code changes; the attempt lifecycle already assumes evaluation is asynchronous and can fail.
- **Cache the deterministic half.** It is a pure function of (submission, problem version), so identical resubmissions never re-run.
- **Polling** → **SSE** on the attempt endpoint when concurrent learners make the poll volume matter.
- **The deterministic floor is the load-shedding mechanism.** Under model pressure, stop scheduling the LLM evaluator and every learner still gets an immediate, honest, clearly-marked partial review instead of a queue that grows.